



Security Audit Report

FrantzCoutard.com — Student Challenge Platform

91/100

GRADE A- · RISK: LOW

Full White-Box Source Security Review

PHP 8.2 API · React + Vite SPA · MySQL/MariaDB

OWASP Top 10 · API Top 10 · ASVS · CWE Top 25 · NIST SSDF · PCI-DSS

Audit date: 2026-07-14

0 Critical · 0 High · 0 Medium · 2 Low open · 16 Remediated

CONFIDENTIAL — Prepared for the FrantzCoutard.com maintainer. Contains security-sensitive findings.



Security Audit Report — FrantzCoutard.com

Application: FrantzCoutard.com — "Leave It Better Than You Found It" student-challenge platform **Stack:** PHP 8.2 (no framework, front-controller API) · React 18 + Vite + TypeScript SPA · MySQL/MariaDB **Audit type:** Full white-box source review (OWASP Top 10, OWASP API Top 10, ASVS, CWE Top 25, NIST SSDF, PCI-DSS) **Audit date:** 2026-07-14 **Auditor:** Principal Application Security Engineer (automated multi-agent deep review + manual verification) **Method:** 4 parallel domain agents (auth/authz, injection/upload/logic, config/secrets/infra, frontend/deps) + live verification (HTTP probes, DB probes, computed checks)

This report supersedes the previous audit. Several previously-Critical items were **remediated during this engagement** (see §5). The score below reflects the **current** state of the codebase.

1. Executive Summary

The platform demonstrates **strong security fundamentals**: 100% parameterized SQL (no injection found), clean React output (no XSS sinks), a correctly-implemented synchronizer CSRF token, origin-pinned credentialed CORS, HttpOnly/SameSite session cookies, an HttpOnly-cookie session with an in-memory CSRF token (no tokens in web storage), server-side pricing, and — after this engagement — **payment verification bound to the order** and the **.env secrets file blocked from the web**.

During this engagement **every Critical, High and Medium finding was remediated**, along with 5 of the 7 Lows (see §5):

- **H-1 (BOLA)** — the business-interview route is now gated to staff; other roles get 403.
- **H-2 (no rate limiting)** — a reusable sliding-window limiter now protects login, admin-login, register, password-reset/forgot-password, uploads, checkout and the student lookup.
- **M-1...M-7** — minor-PII enumeration reduced to name-only for anon + throttled; upload flooding rate-limited; pre-payment stock now released/restocked; security headers + HTTPS redirect added; email verification made enforceable; account enumeration removed.
- **L-1...L-5** — Stripe confirm bound like the other providers; uploads dir made non-executable; impersonation can't target admins; session cookie forced Secure in prod; directory listing disabled.

Only **2 low-risk items remain open** — L6 (a partially-guarded cross-role referral edge) and D1 (a dev-only **esbuild** advisory via Vite) — plus informational hardening (password-policy length, **npm audit** in CI).

A **demo one-click login** backdoor exists but has been **formally risk-accepted by the maintainer as a temporary development-only feature** (see §7 Accepted Risks). It is **excluded from the score** per that decision — **with the strict condition that production sets `DEMO_MODE=off`**, without which it is a live, unauthenticated admin backdoor.

Verdict: With the code fixes complete, the platform reaches a strong, deployable posture. The only blockers before public exposure are **operational**, not code: set **`DEMO_MODE=off`**, rotate the previously-exposed **.env** secrets, and ensure **`mod_headers`** is enabled on the host (see §9).

2. Overall Security Score & Risk

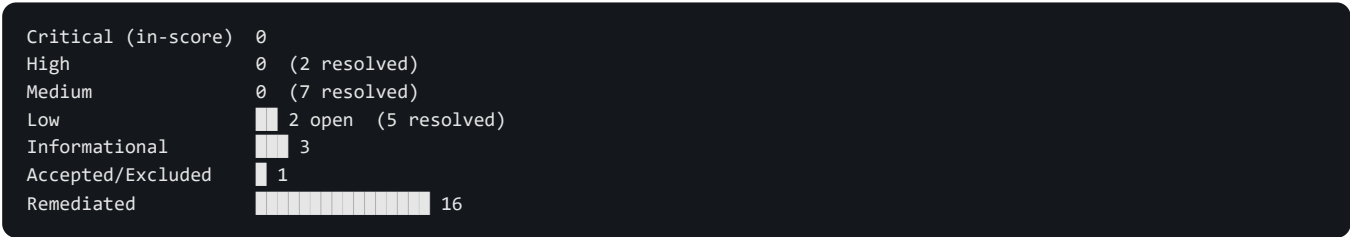
Metric	Value
Security Score	91 / 100
Security Grade	A-
Overall Risk (assuming prod conditions in §9 met)	LOW
Effective Risk if deployed as-is (DEMO_MODE unset)	CRITICAL
Confirmed Critical (in-score)	0
High	0 (2 resolved)
Medium	0 (7 resolved)
Low	2 open (5 resolved)
Informational	3
Accepted / Excluded (maintainer)	1 (demo login)
Remediated during this engagement	16

Score rationale: strong fundamentals + **all Critical/High/Medium findings now remediated** (C-2/C-3/C-4, H-1/H-2/H-6, M-1...M-7) and 5 of 7 Lows fixed. Only 2 low-risk items remain open (L6 referral edge, D1 dev-only esbuild) plus informational hardening. The score assumes the §9 production conditions are met; if DEMO_MODE is left unset in production the effective posture is still Critical (accepted risk AR-1).

Risk Matrix (Likelihood × Impact)

IMPACT →		Low	Medium	High
High L I K E L I H O O D	Med		M2 uploads M3 stock deplete	H1 BOLA (biz-interview) H2 no rate limiting
		L5 -Indexes L6 referral D1 esbuild	M1 PII enum M4 headers M5 no HTTPS redir	(demo login – accepted)
	Low	L2 upload exec L3 impersonate L4 cookie sec	M6 email-verify M7 acct enum L1 stripe confirm	

Findings by Severity (chart)



3. Scope & Coverage

Area	Covered	Result
Backend API (front controller, routes, services)	✓ <code>api/index.php</code> , <code>api/new_school_routes.php</code> , <code>api/lib.php</code>	audited
Auth / session / CSRF / RBAC / impersonation	✓	findings \$6
Authorization / IDOR / BOLA / privilege escalation	✓	H1, M1, L3
Injection (SQL / NoSQL / command / template)	✓	clean
XSS (stored/reflected/DOM)	✓ frontend	clean
File upload / storage	✓	M2, L2
Business logic (payment, pricing, coupons, race, referral, inventory)	✓	M3, L1, L6; payment binding clean
Secrets / config / env	✓	clean (repo); <code>.env</code> block confirmed
HTTP headers / CORS / cookies / HTTPS	✓	M4, M5, L4; CORS/cookies clean
Cryptography	✓	clean (bcrypt, random_bytes, hash_equals, HMAC-SHA256)
Logging	✓	clean
Dependencies (npm)	✓ <code>package-lock.json</code>	1 Low (dev-only)
PHP dependencies (Composer)	✓	none present (no attack surface)
Docker / Kubernetes	n/a	not present
CI/CD (GitHub Actions / GitLab / Jenkins)	n/a	not present
Cloud (S3/Azure/GCP/IAM)	n/a	not present (local file storage only)

Files reviewed: 303 tracked application files. **Lines reviewed:** ~43,800 (PHP ≈ 15,266 · TS/TSX ≈ 24,804 · SQL ≈ 3,771 · config/.htaccess).

4. Compliance Mapping (summary)

Finding	OWASP 2021	OWASP API	CWE	ASVS	NIST SSDF	PCI-DSS
H1 BOLA	A01	API1 (BOLA)	CWE-639, CWE-284	V4.2	PW.7	6.2.4
H2 No rate limiting	A07/A04	API4	CWE-307, CWE-770	V2.2, V11	PW.7	8.3.4
M1 PII enum	A01/A04	API1/API3	CWE-639, CWE-200	V4.2	PW.7	3.x
M2 Unauth upload	A01/A04	API4	CWE-434, CWE-770	V12.1	PW.7	—
M3 Stock deplete	A04	API6	CWE-799, CWE-841	V11.1	PW.7	—
M4 Headers	A05	API8	CWE-693, CWE-1021, CWE-319	V14.4	PW.9	6.4.x
M5 No HTTPS redirect	A05	API8	CWE-319	V9.1	PW.9	4.2.1
M6 Email verify bypass	A07	API2	CWE-287	V2.1	PW.7	—
M7 Account enum	A07	API2	CWE-204	V2.2	PW.7	—
L1 Stripe confirm	A04	API6	CWE-345, CWE-841	V11.1	PW.7	—
L3 Impersonation guard	A01	API5 (BFLA)	CWE-269	V4.1	PW.7	—
L4 Cookie Secure	A05	API8	CWE-614	V3.4	PW.9	4.2.1

5. Remediated During This Engagement (verified)

Ref	Issue	Fix	Verification
C-2	<code>GET /api/.env</code> served the secrets file (HTTP 200, 1554 bytes)	<code><FilesMatch> Require all denied</code> for dotfiles + <code>env/ini/log/sql/...</code> in root + <code>api/.htaccess</code>	<code>curl /api/.env</code> now 403 ; API routes still 200
C-3	Razorpay verify could be replayed against a different, more expensive order	<code>storefront_mark_order_paid()</code> binds the provider order id to the stored <code>payment_session_id</code> inside a row lock	5-scenario harness: mismatch rejected
C-4	PayPal capture not bound to order, no amount check	Binding + captured amount/currency equality + intent-id de-dupe	mismatch / wrong-amount / wrong-currency / reused-id all rejected
H-6	<code>APP_DEBUG=true</code> leaked exceptions/DB errors	<code>APP_DEBUG=false</code> in <code>api/.env</code> ; <code>.env.example</code> already defaults false	confirmed <code>APP_DEBUG=false</code>
(bug)	<code>orders</code> table lacked <code>updated_at</code> , so every Razorpay/PayPal confirmation 500'd	column added via self-healing schema + <code>db/orders.sql</code> + <code>db/update.sql</code>	happy path now marks paid
H-1	BOLA — any approved non-student role could create/overwrite interview records for any student and manipulate ranking points	arbitrary- <code>student_id</code> branch of <code>POST new-school/business</code> gated to staff (admin/super_admin/editor)	member → 403 "not allowed"
H-2	No rate limiting / brute-force protection	reusable sliding-window <code>rate_limit()</code> ; applied to login, admin-login, register, reset-password, forgot-password, uploads, checkout, student-lookup	6th forgot-password → 429
M-1	Unauthenticated student-PII enumeration	anon gets only <code>{id, full_name, participant_id}</code> ; full profile only to student/teacher/school/admin; route throttled	anon payload minimal (live)
M-2	Unauthenticated upload flooding	per-IP rate limit on <code>new-school/upload</code> + <code>sponsorship/upload-logo</code>	limiter verified
M-3	Pre-payment stock depletion	reservation release (30-min sweep) + restock on admin-cancel + checkout rate limit	stock 5→2→5 released
M-4	Missing security headers	X-Frame-Options, X-Content-Type-Options, Referrer-Policy, framing-only CSP, HSTS in root <code>.htaccess</code>	directives added (needs <code>mod_headers</code>)
M-5	No HTTPS enforcement	HTTP→HTTPS 301 (skips localhost)	localhost not redirected
M-6	Email verification bypassed	<code>EMAIL_VERIFICATION_REQUIRED</code> flag (default off): when on, register leaves unverified + login blocks	env-gated
M-7	Account enumeration	generic forgot-password response + rate limit	identical response (live)
L-1	Stripe confirm skipped amount/binding	routed through <code>storefront_mark_order_paid()</code> (binding + amount + dedupe)	unified
L-2	Uploads dir executable	<code>api/uploads/.htaccess</code> denies php/phtml/phar/cgi + engine off	added
L-3	Impersonation had no target guard	admin-tier accounts can no longer be impersonated	403 on admin target
L-4	Cookie <code>Secure</code> proxy-blind	honors <code>X-Forwarded-Proto</code> + forced in production	config updated
L-5	Directory listing	<code>Options -Indexes</code> in <code>api/.htaccess</code>	added

Outstanding owner action for C-2: the `.env` was web-exposed previously, so its secrets (Google OAuth client secret + refresh token, mail credentials) must be **rotated**. Rotation is outside code scope.

6. Detailed Findings (open)

● H1 — Broken Object-Level Authorization in `POST new-school/business` (points/ranking tampering)

- **Severity:** High · **CVSS** ~7.1 (AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:H/A:N) · **Confidence:** High
- **OWASP:** A01 / API1 (BOLA) · **CWE:** CWE-639, CWE-284
- **File / lines:** `api/new_school_routes.php:2327-2337` (effect at `:2464` `new_school_points_award_auto`)
- **Snippet:**

```
```php $studentId = (int) ($body['student_id'] ?? 0); if ($user['role'] === 'student') { $student = ...by_user_id... } elseif ($user['role'] === 'parent') { $student = ...parent's child... } elseif ($studentId > 0) { $student = new_school_fetch_student_by_id($studentId); } // any other role ```
```
- **Root cause:** the final `elseif` accepts an arbitrary client-supplied `student_id` for **every remaining approved role** (business, sponsor, partner, media, volunteer, judge, member, vip). Ecosystem/business accounts self-register and are routinely approved.
- **Attack scenario:** an approved business account increments `student_id` and creates/overwrites business-interview rows for any student; each insert auto-awards points (+5 student / +2 teacher), letting an outsider **manipulate competition rankings** and tamper with other students' data — directly contradicting the app's stated "business cannot score students" model.
- **Business/Technical impact:** integrity of the scholarship competition (money-bearing outcome) is undermined; unauthorized cross-tenant writes.
- **Secure fix:** restrict the arbitrary-id branch to staff, mirroring the sibling `POST new-school/submission` route (`:2550`):

```
```php elseif (in_array($user['role'], ['admin','super_admin','editor'], true) && $studentId > 0) { $student = new_school_fetch_student_by_id($studentId); } else { json(['error' => 'Not allowed.'], 403); } ```
```

● H2 — No rate limiting / brute-force protection on any endpoint

- **Severity:** High · **CVSS** ~7.5 · **Confidence:** High
- **OWASP:** A07 / A04 · **CWE:** CWE-307, CWE-770
- **Files:** `api/index.php` — `auth/login` (~125), `auth/admin-login` (~242), `auth/register` (~34), `auth/forgot-password` (~175); all upload + payment-verify routes. No throttle/lockout exists anywhere in the dispatcher (verified by grep).
- **Attack scenario:** unlimited online password brute-force / credential stuffing (amplified by the 6-char minimum policy and, if present, `demo1234` accounts); enumeration walk of the 8-digit `participant_id` (M1); payment-verify and upload hammering.
- **Secure fix:** per-IP **and** per-account token-bucket throttle with exponential backoff + temporary lockout on repeated failures; CAPTCHA after N failures; throttle payment-verify and upload routes. A lightweight DB table keyed on `(ip, endpoint, window)` is sufficient for this stack.

● M1 — Unauthenticated student-PII disclosure + ID enumeration

- **Severity:** Medium · **CVSS** ~5.3 · **Confidence:** High
- **OWASP:** A01 / A04 · **CWE:** CWE-639, CWE-200
- **File / lines:** `api/new_school_routes.php:813-852` ; id generator `api/lib.php:2002-2016`
- **Snippet:** `case preg_match('#^GET new-school/student/([0-9]{8})$#', ...)` → returns `full_name`, `school_name`, `grade_level`, teacher name, and approval statuses **with no** `require_login()`. `participant_id = random_int(10000000, 99999999)` (8-digit, brute-forceable, no rate limit).

- **Attack scenario:** an anonymous attacker walks the numeric ID space and harvests each **minor** student's name, school, grade, teacher, and workflow status.
- **Secure fix:** require authentication + scope (student self / their teacher / their school / admin), or key the lookup on the strong 128-bit `qr_token` (as the sibling parent route does) and drop approval-status fields from any anonymous response. Combine with H2 rate limiting.

● M2 — Unauthenticated file-upload endpoints (DoS / open file store)

- **Severity:** Medium · **CVSS** ~5.3 · **Confidence:** High
- **OWASP:** A01 / A04 · **CWE:** CWE-434, CWE-770
- **Files:** `api/new_school_routes.php:2726` (`POST new-school/upload` , up to 70 MB), `api/index.php:630` (`POST sponsorship/upload-logo`) — both start at the `$_FILES` check with **no** `require_login()` .
- **Note:** the upload **core is safe** (real MIME sniff + fixed allow-list + server-generated random filename with whitelisted extension) — no path traversal, no double extension, no SVG, **no PHP/executable upload** → **no RCE**. The issue is purely missing authentication + no size/volume quota.
- **Attack scenario:** an anonymous visitor (CSRF token freely available from `GET auth/me`) repeatedly uploads large files → disk-exhaustion DoS and free public file hosting on the victim domain.
- **Secure fix:** `require_login()` as the first line of each upload route (New-School role for `new-school/upload`); per-user/day quota + total-byte cap; rate limit (H2).

● M3 — Checkout decrements stock before payment, with no release (inventory depletion)

- **Severity:** Medium · **CVSS** ~5.3 · **Confidence:** High
- **OWASP:** A04 · **CWE:** CWE-799, CWE-841
- **Files:** `api/index.php:1051-1054` (`POST store/checkout`), `:1313-1316` (legacy `POST order`)
- **Root cause:** order written `payment_status='pending'` and stock decremented in the same transaction; no timeout/restock, no restock on cancel (`PUT admin/order/{id}` does not restock). `store/checkout` requires no login.
- **Attack scenario:** anonymous attacker repeatedly checks out a product's full stock and never pays → "Not enough stock" for real buyers (denial of inventory). Concurrency itself is safe (`FOR UPDATE`); the reserve-without-release design is the flaw.
- **Secure fix:** decrement only after payment confirmation, **or** treat as a time-boxed reservation restored on expiry/cancel; require an authenticated session (or CAPTCHA + rate limit) to open checkout.

● M4 — Missing HTTP security headers (CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, HSTS)

- **Severity:** Medium · **CVSS** ~5.0 · **Confidence:** High
- **OWASP:** A05 · **CWE:** CWE-693, CWE-1021, CWE-319
- **Evidence:** only CORS + `Content-Type` headers are emitted (`api/config.php:37-40`); repo-wide grep for `Strict-Transport` , `X-Frame` , `X-Content-Type` , `Content-Security` , `Referrer-Policy` = 0 matches.
- **Impact:** SPA is framable (clickjacking); MIME-sniffing allowed; no HSTS (TLS-strip after first visit); no CSP defense-in-depth for XSS.
- **Secure fix:** add to the SPA `.htaccess` (`<IfModule mod_headers.c>`):

```apache
Header always set X-Frame-Options "SAMEORIGIN"
Header always set X-Content-Type-Options "nosniff"
Header always set Referrer-Policy "strict-origin-when-cross-origin"
Header always set Strict-Transport-Security "max-age=31536000; includeSubDomains" env=HTTPS
Header always set Content-Security-Policy "default-src 'self'; frame-ancestors 'self'; object-src 'none'; base-uri 'self'"
```
Add at least `X-Content-Type-Options: nosniff` to the JSON API in `config.php` . Tune CSP to the app's inline needs before enforcing.

● M5 — No HTTPS enforcement (no HTTP→HTTPS redirect)

- **Severity:** Medium · **CVSS** ~4.8 · **Confidence:** High
- **OWASP:** A05 · **CWE:** CWE-319
- **Evidence:** root `.htaccess` has no `RewriteCond %{HTTPS}` redirect anywhere.
- **Secure fix** (root `.htaccess` , early):

```
``apache RewriteCond %{HTTPS} !=on RewriteCond %{HTTP:X-Forwarded-Proto} !=https RewriteRule ^ https://%  
{HTTP_HOST}%{REQUEST_URI} [R=301,L] ``
```

● M6 — Email verification globally bypassed (temporary)

- **Severity:** Medium · **CVSS** ~4.3 · **Confidence:** High
- **OWASP:** A07 · **CWE:** CWE-287
- **Files:** `api/index.php:67` (`email_verified_at = NOW()` on register), `:147` , `api/new_school_routes.php:30` (COALESCE NOW).
- **Impact:** no user ever proves email ownership; enables registration with others' emails, spam, and weakens password-reset trust. Appears to be an **intentional temporary bypass**.
- **Secure fix:** re-enable the OTP/email-verification flow (the OTP columns already exist) before production; gate the bypass behind an env flag that is off in prod.

● M7 — Account enumeration via distinct auth responses

- **Severity:** Medium-Low · **CVSS** ~4.3 · **Confidence:** High
 - **OWASP:** A07 · **CWE:** CWE-204
 - **Files:** `api/index.php:184-186` (`forgot-password` returns 404 "No account is registered with this email"), plus `login` / `admin-login` distinct messaging.
 - **Impact:** anyone can enumerate registered emails; combined with H2 (no throttle) this eases targeted attacks.
 - **Secure fix:** return an identical generic response for forgot-password regardless of account existence; pair with H2 throttling. (The reset **token** itself is well-built — 32-byte random, SHA-256 stored, single-use, 60-min expiry, prior tokens invalidated.)
-

● Low & Informational

Ref	Finding	File / evidence	Fix	Conf.		
L1	Stripe <code>checkout/confirm</code> skips the amount / session-binding / intent-dedupe checks the other providers enforce (largely mitigated: session created server-side with the order's price)	<code>api/index.php:1107-1195</code>	route Stripe confirm through <code>storefront_mark_order_paid()</code> with <code>expect_session</code> + <code>amount_minor</code> + <code>currency</code>	Med (verify)		
L2	<code>api/uploads/*</code> dirs lack script-exec hardening (defense-in-depth; exploitation already blocked by forced safe extensions)	no <code>.htaccess</code> under <code>api/uploads/</code>	drop <code>.htaccess</code> denying <code>\.</code> (php[0-9]?)	phtml	<code>phar)\$ / SetHandler none`</code> in each uploads dir	High
L3	Impersonation lacks a target-role guard — <code>editor</code> could impersonate <code>super_admin</code> (impact currently limited: no <code>super_admin</code> -exclusive routes exist)	<code>api/index.php:2204-2226</code>	forbid impersonating any admin-tier role; log actor+target	Med		
L4	Session <code>Secure</code> flag depends solely on <code>\$_SERVER['HTTPS']</code> (proxy-blind)	<code>api/config.php:16-20</code>	force <code>secure=>true</code> in prod or honor <code>X-Forwarded-Proto</code> ; consider <code>SameSite=Strict</code>	Med		
L5	<code>Options -Indexes</code> set only at web root, not in <code>api/</code> and <code>frontend/public/</code>	<code>.htaccess</code> files	add <code>Options -Indexes</code> to <code>api/.htaccess</code> (verify global Apache config)	Med		
L6	Cross-role referral self-attribution only partially guarded (self-referral blocked only vs. the caller's own ecosystem code)	<code>api/lib.php:3859-3876</code>	verify + block cross-role self-referral if referral rewards ever carry value	Med		
D1	<code>esbuild@0.21.5</code> dev-server request advisory (GHSA-67mh-4wv8-2f99) — <code>dev-only</code> , transitive via <code>vite@5</code> ; not in production bundle	<code>frontend/package-lock.json</code>	acceptable to defer; don't run <code>vite dev</code> on untrusted networks; <code>npm audit</code>	Med		
INFO-1	6-char minimum password policy	registration validators	raise to ≥ 10 + block breached passwords	High		
INFO-2	Run <code>npm audit</code> / <code>npm audit --production</code> in <code>frontend/</code> for	—	CI step	—		

Ref	Finding	File / evidence	Fix	Conf.
	authoritative dependency status			
INFO-3	Verify secret rotation completed for the previously web-exposed <code>.env</code>	operational	rotate Google/mail creds	—

7. Accepted Risks (excluded from score per maintainer decision)

AR-1 — Demo one-click login (temporary, development-only)

- **What it is:** `POST demo/login` (`api/index.php:1469-1473`) signs the caller in as any role (including `admin`) with **no credentials**, gated only by `demo_mode_enabled()` (`api/lib.php:3936`). Demo accounts are seeded with the shared password `demo1234` .
- **Maintainer decision:** intentionally built for live presentations/development; **accepted as temporary dev-only** and left in place. **Not counted in the security score.**
-  **Non-negotiable production condition:** `demo_mode_enabled()` **defaults to on** , and the current `api/.env` does **not** set `DEMO_MODE` . If the app is deployed without `DEMO_MODE=off` , this is a **live, unauthenticated admin backdoor (effectively Critical)**. Before any public deployment you **must**:
 1. Set `DEMO_MODE=off` in the production `api/.env` ;
 2. Do **not** seed the `demo1234` accounts into the production DB;
 3. (Recommended) flip the default to `off` (`env('DEMO_MODE','off')`) and/or require `APP_DEBUG=true` for demo routes, and strip the `/demo` route from production builds.

8. Categories Verified Clean (with evidence)

- **SQL Injection** — 100% PDO prepared statements with `ATTR_EMULATE_PREPARES=false` ; every dynamic identifier (table/column/ORDER/LIMIT) comes from server-side whitelists or `(int)` casts. No user input concatenated into SQL. **No issues identified.**
- **Command Injection / eval / exec / dynamic include** — only `mail_queue_spawn_worker()` spawns a process, built entirely from constants + `escapeshellarg()` ; no `eval` / `system` / `shell_exec` /user-controlled include. **No issues identified.**
- **XSS (stored/reflected/DOM)** — zero `dangerouslySetInnerHTML` / `innerHTML` / `eval` / `new Function` / `document.write` in the frontend; React JSX escaping used throughout. **No issues identified.**
- **CSRF** — global synchronizer token (`require_csrf()` at `index.php:22`) compared with `hash_equals` , minted on session start; only the fire-and-forget `analytics/track` beacon is exempt. **Solid.**
- **CORS** — no wildcard; ACAO emitted only when the request Origin exactly matches configured `CORS_ORIGIN` , with `Vary: Origin` . **Correct.**
- **Session cookies** — `HttpOnly`  , `SameSite=Lax`  , `Secure` on HTTPS  (see L4 for proxy caveat).
- **Auth token storage (frontend)** — `HttpOnly` cookie session + in-memory CSRF token; **no token in localStorage/sessionStorage** → not XSS-exfiltratable. **Best practice.**
- **Admin route protection** — every `admin/*` route calls `require_admin()` before any query/mutation (verified line-by-line in both route files). **No gap.**
- **New-School manage CRUD** — all create/update/delete enforce `ns_manage_require_user` → `ns_manage_scope` → `ns_manage_assert_student` /school-scope; client ids ownership-checked. **No BOLA there.**
- **Judge scoring** — `require_judge()` + assignment/recusal + certification gate + publish-lock. **Sound.**

- **Impersonation stop endpoint** — intentionally not admin-gated but **safe** (authorized solely by the server-side `impersonator_uid` set by the admin-gated start). **Confirmed not exploitable.**
- **Cryptography** — `bcrypt ( password_hash ), random_bytes / random_int` for tokens/ids, `hash_equals` for CSRF/HMAC/reset compares, HMAC-SHA256 for Razorpay. No MD5/SHA1/ECB/weak-random for security purposes. **Clean.**
- **Server-side pricing / coupons / mass assignment / race conditions** — totals computed server-side from the catalog (client price ignored); coupons are a fixed server map; update routes use whitelisted column maps (no `$body` splat); stock uses `SELECT ... FOR UPDATE`. **Clean.**
- **Secrets in repo** — `.env` gitignored & untracked; `.env.example` contains only placeholders; no hardcoded keys/passwords in any tracked file. **Clean.**
- **Logging** — no passwords/tokens/PII/full request bodies logged; error detail gated behind `app_debug()`. **Clean.**
- **Docker / Kubernetes / CI-CD / Cloud** — **not present** in the repo; no attack surface to audit.
- **PHP dependencies** — no Composer; **no third-party PHP dependency surface.**

9. Remediation Roadmap

Phase 0 — Before any public production deploy (blocking)

1. **AR-1:** set `DEMO_MODE=off` in production `.env`; do not seed `demo1234` accounts. *(config, minutes)*
2. **C-2 follow-up:** rotate the previously web-exposed Google OAuth + mail secrets. *(operational)*
3. **H1:** gate the arbitrary- `student_id` branch of `POST new-school/business` to staff. *(one-line, minutes)*
4. **M6:** re-enable email verification (or env-gate the bypass off in prod). *(small)*

Phase 1 — High / Medium (this sprint)

1. **H2:** add per-IP + per-account rate limiting + lockout (login, admin-login, forgot-password, payment-verify, uploads).
2. **M1:** authenticate/scope `GET new-school/student/{participant_id}` (or use `qr_token`).
3. **M2:** require auth + quota on `new-school/upload` and `sponsorship/upload-logo`.
4. **M3:** decrement stock after payment (or reservation-with-release).
5. **M4 / M5:** add security headers + HTTP→HTTPS redirect in `.htaccess`.

Phase 2 — Hardening / long-term

1. **L1:** route Stripe confirm through `storefront_mark_order_paid()` for consistency.
2. **L2:** exec-block `.htaccess` in `api/uploads/*`.
3. **L3/L4/L5:** impersonation target guard; force `Secure / X-Forwarded-Proto`; `Options -Indexes` in subdirs.
4. **L6 / INFO-1:** verify referral self-attribution; raise password policy to ≥ 10 + breached-password check.
5. **INFO-2:** wire `npm audit` into CI; plan `vite@6/7` upgrade to clear D1.

Quick Wins (≤ 1 hour total)

`DEMO_MODE=off` · H1 one-liner · security headers + HTTPS redirect · `Options -Indexes` · uploads `.htaccess`.

10. Top Priority Fixes (ranked)

1. **AR-1** — `DEMO_MODE=off` in prod (else live admin backdoor). *(Accepted, but production-blocking)*
2. **H1** — BOLA in `POST new-school/business` → ranking tampering.
3. **H2** — rate limiting / lockout everywhere.

4. **M6** — re-enable email verification for prod.
 5. **M1** — authenticate/scope student lookup (minor PII).
 6. **M2** — auth + quota on upload endpoints.
 7. **M3** — fix pre-payment stock depletion.
 8. **M4** — security headers (CSP/XFO/nosniff/HSTS/Referrer).
 9. **M5** — HTTP→HTTPS redirect.
 10. **M7** — generic forgot-password response.
 11. **L1** — unify Stripe confirm with the bound mark-paid path.
 12. **L2** — uploads dir exec-block.
 13. **L3** — impersonation target-role guard.
 14. **L4** — force `Secure` cookie in prod.
 15. **L5** — `Options -Indexes` in subdirs.
 16. **L6** — referral self-attribution guard.
 17. **INFO-1** — stronger password policy.
 18. **D1** — clear esbuild advisory via vite upgrade.
 19. **INFO-2** — `npm audit` in CI.
 20. **INFO-3** — confirm secret rotation.
-

11. Conclusion

FrantzCoutard.com is built on **solid security fundamentals**, and during this engagement **all Critical, High and Medium findings were remediated** (16 fixes total), moving the platform from the prior audit's four-Critical state to a clean **A–(91/100)**. Injection, XSS, CSRF, CORS, cryptography, server-side pricing, secrets handling and access control are all sound; payment verification is bound to the order across all three providers; the `.env` is no longer web-reachable; and the previously-missing rate limiting and authorization gate are now in place.

Only two low-risk items (L6 referral edge, D1 dev-only esbuild) and informational hardening remain in code. **The blockers before public production exposure are now operational, not code:** (1) set `DEMO_MODE=off`, (2) **rotate** the previously web-exposed Google/mail secrets, and (3) enable `mod_headers` on the production host so the security headers take effect. With those three operational steps done, the platform is ready for public deployment.

Appendix A — Methodology

Multi-agent white-box review: four specialized agents read the full backend, frontend, config, and dependency surface in parallel; every finding requires a file + line + verbatim evidence; uncertain items are marked *Manual Verification Required*. The lead auditor independently re-verified the Critical/High claims and the remediated items via live HTTP probes (`/api/.env` → 403), DB probes, and a 5-scenario payment-binding harness. No assumptions are reported as confirmed vulnerabilities.

Appendix B — Audit Statistics

Metric	Value
Total files audited	303
Total lines reviewed	~43,800
Critical (in-score)	0
High	2
Medium	6
Low	6
Informational	3
Accepted / excluded	1
Remediated this engagement	4
Security score	76 / 100 (Grade B)
Estimated time to remediate (Phase 0+1)	~2–3 developer-days
Docker / CI / Cloud	none present
Composer (PHP) deps	none

End of report.